Custom CUDA-Optimized GEMM on Tesla T4 (Google Colab)

NVIDIA DevTech Internship Showcase

This notebook demonstrates building, benchmarking, and profiling custom CUDA GEMM kernels (Naive vs. Tiled Shared Memory vs. cuBLAS) on an NVIDIA Tesla T4 GPU.

Step 1: Verify NVIDIA GPU Allocation

Ensure your Colab runtime is set to **GPU** (`Runtime -> Change runtime type -> T4 GPU`).

```
!nvidia-smi
```

```
Wed Oct 14 17:11:49 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   45C    P8             10W /   70W |       0MiB /  15360MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

```
!nvcc --version
```

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Wed_Aug_20_01:58:59_PM_PDT_2025
Cuda compilation tools, release 13.0, V13.0.88
Build cuda_13.0.r13.0/compiler.36424714_0
```

Step 2: Write or Verify `gemm.cu`

If you uploaded `gemm.cu` directly into Colab's file pane (left sidebar), skip this cell. Otherwise, run the cell below to write `gemm.cu` directly.

```
!ls -lh gemm.cu
```

```
ls: cannot access 'gemm.cu': No such file or directory
```

```
%%writefile gemm.cu
#include <iostream>
#include <iomanip>
#include <vector>
#include <cmath>
#include <cuda_runtime.h>
#include <cublas_v2.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err = call;                                             \
        if (err != cudaSuccess) {                                           \
            std::cerr << "CUDA Error at " << __FILE__ << ":" << __LINE__   \
                      << " - " << cudaGetErrorString(err) << std::endl;     \
            exit(EXIT_FAILURE);                                             \
        }                                                                   \
    } while (0)

#define CUBLAS_CHECK(call)                                                  \
    do {                                                                    \
        cublasStatus_t status = call;                                       \
        if (status != CUBLAS_STATUS_SUCCESS) {                              \
            std::cerr << "cuBLAS Error at " << __FILE__ << ":" << __LINE__  \
                      << " - Code: " << status << std::endl;                \
```

```
            exit(EXIT_FAILURE);                                                 \
        }                                                                       \
    } while (0)

constexpr int TILE_DIM = 32;

__global__ void gemm_naive(const float* __restrict__ A,
                           const float* __restrict__ B,
                           float* __restrict__ C,
                           int N) {
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    int row = blockIdx.y * blockDim.y + threadIdx.y;

    if (row < N && col < N) {
        float sum = 0.0f;
        #pragma unroll 4
        for (int k = 0; k < N; ++k) {
            sum += A[row * N + k] * B[k * N + col];
        }
        C[row * N + col] = sum;
    }
}

__global__ void gemm_tiled(const float* __restrict__ A,
                           const float* __restrict__ B,
                           float* __restrict__ C,
                           int N) {
    const int tx = threadIdx.x;
    const int ty = threadIdx.y;
    const int bx = blockIdx.x;
    const int by = blockIdx.y;

    const int row = by * TILE_DIM + ty;
    const int col = bx * TILE_DIM + tx;

    __shared__ float s_A[TILE_DIM][TILE_DIM];
    __shared__ float s_B[TILE_DIM][TILE_DIM];

    float sum = 0.0f;
    const int numTiles = (N + TILE_DIM - 1) / TILE_DIM;

    for (int t = 0; t < numTiles; ++t) {
        int a_col = t * TILE_DIM + tx;
        int b_row = t * TILE_DIM + ty;

        s_A[ty][tx] = (row < N && a_col < N) ? A[row * N + a_col] : 0.0f;
        s_B[ty][tx] = (b_row < N && col < N) ? B[b_row * N + col] : 0.0f;

        __syncthreads();

        #pragma unroll
        for (int k = 0; k < TILE_DIM; ++k) {
            sum += s_A[ty][k] * s_B[k][tx];
        }

        __syncthreads();
    }

    if (row < N && col < N) {
        C[row * N + col] = sum;
    }
}

void run_cublas_gemm(cublasHandle_t handle,
                     const float* d_A,
                     const float* d_B,
                     float* d_C,
                     int N) {
    const float alpha = 1.0f;
    const float beta = 0.0f;
    CUBLAS_CHECK(cublasSgemm(
        handle,
        CUBLAS_OP_N, CUBLAS_OP_N,
        N, N, N,
        &alpha,
        d_B, N,
        d_A, N,
        &beta,
        d_C, N
    ));
}
```

```
bool verify_results(const float* ref, const float* test, int N, float tolerance = 1e-3f) {
    float max_diff = 0.0f;
    int mismatch_count = 0;
    for (int i = 0; i < N * N; ++i) {
        float diff = std::fabs(ref[i] - test[i]);
        if (diff > max_diff) max_diff = diff;
        if (diff > tolerance) mismatch_count++;
    }
    return (mismatch_count == 0);
}

struct BenchmarkResult {
    std::string name;
    float time_ms;
    float gflops;
    float speedup;
    bool verified;
};

void run_benchmark_for_size(int N, cublasHandle_t cublas_handle) {
    std::cout << "\n" << std::string(80, '=') << std::endl;
    std::cout << " BENCHMARKING MATRIX MULTIPLICATION: N = " << N
              << " (" << N << " x " << N << " = " << (size_t)N * N * sizeof(float) / (1024 * 1024)
              << " MB per matrix)" << std::endl;
    std::cout << std::string(80, '=') << std::endl;

    size_t bytes = (size_t)N * N * sizeof(float);
    double total_flops = 2.0 * (double)N * (double)N * (double)N;

    std::vector<float> h_A(N * N), h_B(N * N), h_C_cublas(N * N), h_C_test(N * N);
    for (int i = 0; i < N * N; ++i) {
        h_A[i] = static_cast<float>(rand() % 100) / 100.0f;
        h_B[i] = static_cast<float>(rand() % 100) / 100.0f;
    }

    float *d_A = nullptr, *d_B = nullptr, *d_C = nullptr;
    CUDA_CHECK(cudaMalloc(&d_A, bytes));
    CUDA_CHECK(cudaMalloc(&d_B, bytes));
    CUDA_CHECK(cudaMalloc(&d_C, bytes));

    CUDA_CHECK(cudaMemcpy(d_A, h_A.data(), bytes, cudaMemcpyHostToDevice));
    CUDA_CHECK(cudaMemcpy(d_B, h_B.data(), bytes, cudaMemcpyHostToDevice));

    cudaEvent_t start, stop;
    CUDA_CHECK(cudaEventCreate(&start));
    CUDA_CHECK(cudaEventCreate(&stop));

    const int WARMUP_RUNS = 2;
    const int BENCHMARK_RUNS = 5;
    std::vector<BenchmarkResult> results;

    // 1. cuBLAS
    {
        for (int i = 0; i < WARMUP_RUNS; ++i) run_cublas_gemm(cublas_handle, d_A, d_B, d_C, N);
        CUDA_CHECK(cudaDeviceSynchronize());

        CUDA_CHECK(cudaEventRecord(start));
        for (int i = 0; i < BENCHMARK_RUNS; ++i) run_cublas_gemm(cublas_handle, d_A, d_B, d_C, N);
        CUDA_CHECK(cudaEventRecord(stop));
        CUDA_CHECK(cudaEventSynchronize(stop));

        float total_ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&total_ms, start, stop));
        float avg_ms = total_ms / BENCHMARK_RUNS;
        float gflops = (float)((total_flops * 1e-9) / (avg_ms * 1e-3));

        CUDA_CHECK(cudaMemcpy(h_C_cublas.data(), d_C, bytes, cudaMemcpyDeviceToHost));
        results.push_back({"cuBLAS (NVIDIA Library)", avg_ms, gflops, 1.0f, true});
    }

    // 2. Naive
    {
        dim3 blockDim(TILE_DIM, TILE_DIM);
        dim3 gridDim((N + blockDim.x - 1) / blockDim.x, (N + blockDim.y - 1) / blockDim.y);

        for (int i = 0; i < WARMUP_RUNS; ++i) gemm_naive<<<gridDim, blockDim>>>(d_A, d_B, d_C, N);
        CUDA_CHECK(cudaDeviceSynchronize());

        CUDA_CHECK(cudaEventRecord(start));
        for (int i = 0; i < BENCHMARK_RUNS; ++i) gemm_naive<<<gridDim, blockDim>>>(d_A, d_B, d_C, N);
        CUDA_CHECK(cudaEventRecord(stop));
        CUDA_CHECK(cudaEventSynchronize(stop));
```

```
        float total_ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&total_ms, start, stop));
        float avg_ms = total_ms / BENCHMARK_RUNS;
        float gflops = (float)((total_flops * 1e-9) / (avg_ms * 1e-3));

        CUDA_CHECK(cudaMemcpy(h_C_test.data(), d_C, bytes, cudaMemcpyDeviceToHost));
        bool verified = verify_results(h_C_cublas.data(), h_C_test.data(), N);
        results.push_back({"Naive (Global Memory)", avg_ms, gflops, 1.0f, verified});
    }

    // 3. Tiled
    {
        dim3 blockDim(TILE_DIM, TILE_DIM);
        dim3 gridDim((N + blockDim.x - 1) / blockDim.x, (N + blockDim.y - 1) / blockDim.y);

        for (int i = 0; i < WARMUP_RUNS; ++i) gemm_tiled<<<gridDim, blockDim>>>(d_A, d_B, d_C, N);
        CUDA_CHECK(cudaDeviceSynchronize());

        CUDA_CHECK(cudaEventRecord(start));
        for (int i = 0; i < BENCHMARK_RUNS; ++i) gemm_tiled<<<gridDim, blockDim>>>(d_A, d_B, d_C, N);
        CUDA_CHECK(cudaEventRecord(stop));
        CUDA_CHECK(cudaEventSynchronize(stop));

        float total_ms = 0.0f;
        CUDA_CHECK(cudaEventElapsedTime(&total_ms, start, stop));
        float avg_ms = total_ms / BENCHMARK_RUNS;
        float gflops = (float)((total_flops * 1e-9) / (avg_ms * 1e-3));

        CUDA_CHECK(cudaMemcpy(h_C_test.data(), d_C, bytes, cudaMemcpyDeviceToHost));
        bool verified = verify_results(h_C_cublas.data(), h_C_test.data(), N);
        results.push_back({"Tiled (Shared Memory)", avg_ms, gflops, 1.0f, verified});
    }

    float naive_time = results[1].time_ms;
    for (auto& res : results) res.speedup = naive_time / res.time_ms;

    std::cout << "\n+-------------------------+-------------+---------------+------------+-----------+\n";
    std::cout << "| Implementation          | Time (ms)   | Throughput    | Speedup    | Status    |\n";
    std::cout << "|                         |             | (GFLOPS)      | vs Naive   |           |\n";
    std::cout << "+-------------------------+-------------+---------------+------------+-----------+\n";
    for (const auto& res : results) {
        std::cout << "| " << std::left << std::setw(23) << res.name
                  << " | " << std::right << std::setw(11) << std::fixed << std::setprecision(3) << res.time_ms
                  << " | " << std::right << std::setw(13) << std::fixed << std::setprecision(1) << res.gflops
                  << " | " << std::right << std::setw(9)  << std::fixed << std::setprecision(2) << res.speedup <
                  << " | " << std::left  << std::setw(9)  << (res.verified ? "PASS" : "FAIL")
                  << " |\n";
    }
    std::cout << "+-------------------------+-------------+---------------+------------+-----------+\n";

    CUDA_CHECK(cudaEventDestroy(start));
    CUDA_CHECK(cudaEventDestroy(stop));
    CUDA_CHECK(cudaFree(d_A));
    CUDA_CHECK(cudaFree(d_B));
    CUDA_CHECK(cudaFree(d_C));
}

int main() {
    std::cout << "=== NVIDIA DevTech GEMM Benchmark Suite ===" << std::endl;
    int device_id = 0;
    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, device_id));
    std::cout << "GPU: " << prop.name << " (" << prop.multiProcessorCount << " SMs)" << std::endl;

    cublasHandle_t handle;
    CUBLAS_CHECK(cublasCreate(&handle));

    run_benchmark_for_size(1024, handle);
    run_benchmark_for_size(2048, handle);

    CUBLAS_CHECK(cublasDestroy(handle));
    std::cout << "\n[OK] Benchmark Completed." << std::endl;
    return 0;
}
```

```
Writing gemm.cu
```

Step 3: Compile `gemm.cu`

- `-O3`: Maximum host and device optimization

- `-arch=sm_75`: Targets Tesla T4 (Turing architecture)
- `-lcublas`: Links NVIDIA cuBLAS library

```
!nvcc -O3 -std=c++14 -arch=sm_75 gemm.cu -o gemm -lcublas
!ls -lh gemm
```

```
-rwxr-xr-x 1 root root 995K Oct 14 17:11 gemm
```

Step 4: Run the Benchmark Suite ($N = 1024$ and $N = 2048$)

```
!./gemm
```

```
=== NVIDIA DevTech GEMM Benchmark Suite ===
GPU: Tesla T4 (40 SMs)

================================================================================
 BENCHMARKING MATRIX MULTIPLICATION: N = 1024 (1024 x 1024 = 4 MB per matrix)
================================================================================

+------------------------+------------+---------------+------------+-----------+
| Implementation         | Time (ms)  | Throughput    | Speedup    | Status    |
|                        |            | (GFLOPS)      | vs Naive   |           |
+------------------------+------------+---------------+------------+-----------+
| cuBLAS (NVIDIA Library) |      0.640 |        3354.4 |      9.05x | PASS      |
| Naive (Global Memory)  |      5.796 |         370.5 |      1.00x | PASS      |
| Tiled (Shared Memory)  |      4.119 |         521.4 |      1.41x | PASS      |
+------------------------+------------+---------------+------------+-----------+

================================================================================
 BENCHMARKING MATRIX MULTIPLICATION: N = 2048 (2048 x 2048 = 16 MB per matrix)
================================================================================

+------------------------+------------+---------------+------------+-----------+
| Implementation         | Time (ms)  | Throughput    | Speedup    | Status    |
|                        |            | (GFLOPS)      | vs Naive   |           |
+------------------------+------------+---------------+------------+-----------+
| cuBLAS (NVIDIA Library) |      5.233 |        3283.1 |      6.74x | PASS      |
| Naive (Global Memory)  |     35.267 |         487.1 |      1.00x | FAIL      |
| Tiled (Shared Memory)  |     18.851 |         911.4 |      1.87x | FAIL      |
+------------------------+------------+---------------+------------+-----------+

[OK] Benchmark Completed.
```

Step 5: NVIDIA Nsight Compute (`ncu`) Profiling

NVIDIA Nsight Compute is the industry standard kernel profiler used by DevTech engineers.

Profiling Command A: High-Level Speed of Light (SOL) Metrics

Measures SM compute throughput % and DRAM bandwidth throughput % for both kernels.

```
!ncu --metrics sm__throughput.avg.pct_of_peak_sustained_elapsed,dram__throughput.avg.pct_of_peak_sustained_elapse
```

```
gemm_tiled(const float *, const float *, float *, int) (64, 64, 1)x(32, 32, 1), Context 1, Stream 7, Device 0
  Section: Command line profiler metrics
  ---------------------------------------------------- ----------- ------------
  Metric Name                                          Metric Unit Metric Value
  ---------------------------------------------------- ----------- ------------
  dram__throughput.avg.pct_of_peak_sustained_elapsed           %        12.51
  sm__throughput.avg.pct_of_peak_sustained_elapsed             %        75.36
  ---------------------------------------------------- ----------- ------------

gemm_tiled(const float *, const float *, float *, int) (64, 64, 1)x(32, 32, 1), Context 1, Stream 7, Device 0
  Section: Command line profiler metrics
  ---------------------------------------------------- ----------- ------------
  Metric Name                                          Metric Unit Metric Value
  ---------------------------------------------------- ----------- ------------
  dram__throughput.avg.pct_of_peak_sustained_elapsed           %        12.51
  sm__throughput.avg.pct_of_peak_sustained_elapsed             %        75.35
  ---------------------------------------------------- ----------- ------------

gemm_tiled(const float *, const float *, float *, int) (64, 64, 1)x(32, 32, 1), Context 1, Stream 7, Device 0
  Section: Command line profiler metrics
  ---------------------------------------------------- ----------- ------------
  Metric Name                                          Metric Unit Metric Value
  ---------------------------------------------------- ----------- ------------
  dram__throughput.avg.pct_of_peak_sustained_elapsed           %        12.49
  sm__throughput.avg.pct_of_peak_sustained_elapsed             %        75.34
  ---------------------------------------------------- ----------- ------------
```

Profiling Command B: Memory Workload Analysis & Bank Conflicts

Inspects shared memory access efficiency and bank conflicts.

```
!ncu --metrics l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum,l1tex__data_pipe_lsu_wavefronts_mem_shar
```

```
    l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum                      0
    l1tex__data_pipe_lsu_wavefronts_mem_shared_op_ld.sum                402,653,184
    ----------------------------------------------------------- ----------- ------------


```

Profiling Command C: Export Full Profiler Report

Generates a `.ncu-rep` file that you can download and open in the NVIDIA Nsight Compute GUI on your Mac/PC.

```
!ncu --set full -o gemm_profile_report ./gemm
```

```
=== NVIDIA DevTech GEMM Benchmark Suite ===
==PROF== Connected to process 3433 (/content/gemm)
GPU: Tesla T4 (40 SMs)

================================================================================
 BENCHMARKING MATRIX MULTIPLICATION: N = 1024 (1024 x 1024 = 4 MB per matrix)
================================================================================
==PROF== Profiling "volta_sgemm_128x64_nn" - 0: 0%....50%....100% - 33 passes
==PROF== Profiling "volta_sgemm_128x64_nn" - 1: 0%....50%....100% - 33 passes
==PROF== Profiling "volta_sgemm_128x64_nn" - 2: 0%....50%....100% - 33 passes
==PROF== Profiling "volta_sgemm_128x64_nn" - 3: 0%....50%....100% - 33 passes
==PROF== Profiling "volta_sgemm_128x64_nn" - 4: 0%....50%....100% - 33 passes
==PROF== Profiling "volta_sgemm_128x64_nn" - 5: 0%....50%....100% - 33 passes
==PROF== Profiling "volta_sgemm_128x64_nn" - 6: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_naive" - 7: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_naive" - 8: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_naive" - 9: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_naive" - 10: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_naive" - 11: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_naive" - 12: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_naive" - 13: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_tiled" - 14: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_tiled" - 15: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_tiled" - 16: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_tiled" - 17: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_tiled" - 18: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_tiled" - 19: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_tiled" - 20: 0%....50%....100% - 33 passes

+------------------------+-------------+---------------+------------+-----------+
| Implementation         | Time (ms)   | Throughput    | Speedup    | Status    |
|                        |             | (GFLOPS)      | vs Naive   |           |
+------------------------+-------------+---------------+------------+-----------+
| cuBLAS (NVIDIA Library) |    6494.257 |           0.3 |      1.12x | PASS      |
| Naive (Global Memory)   |    7256.763 |           0.3 |      1.00x | PASS      |
| Tiled (Shared Memory)   |    6631.105 |           0.3 |      1.09x | PASS      |
+------------------------+-------------+---------------+------------+-----------+

================================================================================
 BENCHMARKING MATRIX MULTIPLICATION: N = 2048 (2048 x 2048 = 16 MB per matrix)
================================================================================
==PROF== Profiling "volta_sgemm_128x64_nn" - 21: 0%....50%....100% - 33 passes
==PROF== Profiling "volta_sgemm_128x64_nn" - 22: 0%....50%....100% - 33 passes
==PROF== Profiling "volta_sgemm_128x64_nn" - 23: 0%....50%....100% - 33 passes
==PROF== Profiling "volta_sgemm_128x64_nn" - 24: 0%....50%....100% - 33 passes
==PROF== Profiling "volta_sgemm_128x64_nn" - 25: 0%....50%....100% - 33 passes
==PROF== Profiling "volta_sgemm_128x64_nn" - 26: 0%....50%....100% - 33 passes
==PROF== Profiling "volta_sgemm_128x64_nn" - 27: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_naive" - 28: 0%
==WARNING== Launching the workload is taking more time than expected. If this continues to hang, terminate the p
....50%....100% - 33 passes
==PROF== Profiling "gemm_naive" - 29: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_naive" - 30: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_naive" - 31: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_naive" - 32: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_naive" - 33: 0%....50%....100% - 33 passes
==PROF== Profiling "gemm_naive" - 34: 0%....50%....100% - 33 passes
  PROF   Profiling "gemm_tiled"  35: 0%    50%    100%  33 passes
```